



Translating SUMO-K to Higher-Order Set Theory

Chad E. Brown¹, Adam Pease^{1,2(✉)} , and Josef Urban¹ 

¹ Czech Technical University in Prague, Prague, Czech Republic
apease@articulatesoftware.com

² Parallax Advanced Research, Beavercreek, OH, USA

Abstract. We describe a translation from a fragment of SUMO (SUMO-K) into higher-order set theory. The translation provides a formal semantics for portions of SUMO which are beyond first-order and which have previously only had an informal interpretation. It also for the first time embeds a large common-sense ontology into an interactive theorem proving system. We further extend our previous work in finding contradictions in SUMO from first-order constructs to include a portion of SUMO's higher-order constructs. Finally, using the translation, we can create problems that can be proven using higher-order interactive and automated theorem provers. This is tested in several systems and used to form a corpus of higher-order common-sense reasoning problems.

Keywords: ontology · theorem proving · Megalodon · theorem proving · automated theorem proving · automated reasoning · SUMO

1 Introduction and Motivation

The Suggested Upper Merged Ontology (SUMO) [15,16] is a comprehensive ontology of around 20,000 concepts and 80,000 hand-authored logical statements in a higher-order logic. It has an associated integrated development environment called Sigma [19]¹ that interfaces to theorem provers such as E [22] and Vampire [12]. In previous work on translating SUMO to the TPTP [25] THF (Typed Higher-order Form) [1] format, a syntactic translation to THF was created but did not resolve many aspects of the intended higher-order semantics of SUMO.

In this work, we lay the groundwork for a new translation to a language for higher-order automated theorem provers based on expressing SUMO in higher-order set theory. We believe this will attach to SUMO a stronger set-theoretical interpretation that will allow deciding more queries and provide better intuition for avoiding contradictory formalizations. Once this is done, our plan is to train ENIGMA-style [5–8] query answering and contradiction-finding [23] AITP systems on such SUMO problems and develop autoformalization [9–11,28] methods targeting common-sense reasoning based on SUMO. We believe that this is the most viable path towards common-sense reasoning that is both trainable, but also explainable and verifiable, providing an alternative to language models which come with no formal guarantees.

¹ <https://www.ontologyportal.org>.

1.1 Related Work and Contributions

In earlier work, we described [19] how to translate SUMO to the strictly first-order language of TPTP-FOF [20] and TF0 [17, 18, 26]. SUMO has an extensive type structure and all relations have type restrictions on their arguments. Translation to TPTP FOF involved implementing a sorted (typed) logic axiomatically in TPTP by altering all implications in SUMO to contain type restrictions on any variables that appear.

In [21] 35 SUMO queries were converted into challenge problems for first-order automated theorem provers. In many cases, first-order ATPs can prove the corresponding problem. However, some of the queries involve aspects of SUMO that go beyond first-order representation. For example, one of the queries involves a term-level binder (κ).² Several of the queries also involve *row variables* (also called *sequence variables*), i.e., variables that should be instantiated with a list of terms. We discuss here several such examples to motivate the translation to higher-order set theory. We then embed SUMO into the Megalodon system, providing, to our knowledge, the first representation of a large common-sense ontology within an interactive theorem prover (ITP). We then consider the higher-order problems obtained via the translation. This provides a set of challenge problems for higher-order theorem provers that come from a different source than formalized mathematics or program verification.

The rest of the paper is organized as follows. In Sect. 2 we introduce the SUMO-K fragment of SUMO, an extension of the first-order fragment of SUMO. We also show there examples in SUMO that motivate the extensions. Section 3 describes a translation from SUMO-K into a higher-order set theory. We have constructed interactive proofs of the translated form of 23 SUMO-K queries. We describe several of the proofs in Sect. 4. From the interactive proofs we obtain 4880 ATP problems and we measure the performance of higher-order automated theorem provers on this problem set in Sect. 5. Section 6 describes the planned extensions and Sect. 7 concludes. Our code and problem set are available online.³

2 The SUMO-K Fragment

We define a fragment of SUMO we call SUMO-K. This extends the first-order fragment of SUMO with support for row variables, variable arity functions and relations, and the κ class formation term binder.⁴ Elements of SUMO not included in SUMO-K are temporal, modal and probabilistic operations.

We start by defining SUMO-K *terms*, *spines* (lists of terms) and *formulas*. Formally, we have *standard variables* (x), *row variables* (ρ) and *constants* (c).

² Note that by “term-level binder” we mean a binder that yields a term. By way of contrast, $\forall$ and $\exists$ are formula-level binders. κ is used to form classes in SUMO. Informally, one can think of $\kappa x.\psi$ as the class $\{x|\psi\}$.

³ <http://grid01.ciirc.cvut.cz/~chad/sumo2set-0.9.tgz>.

⁴ SUMO classes should not be confused with set-theoretic classes. Our use of “class” in this paper will always refer to SUMO classes.

We will also have *signed rationals* (q) represented by a decimal expression with finitely many digits (i.e., those rationals expressible in such a way) as terms. We define by mutual recursion the sets of SUMO-K terms t , SUMO-K spines s and SUMO-K formulas ψ as follows:

$$\begin{aligned}
 t &::= x|c|q|(x\ s)|(\kappa\ s)|(\kappa x.\psi)|\text{Real}|\text{Neg}|\text{Nonneg}|(t + t)|(t - t)|(t * t)|(t / t) \\
 s &::= t\ s|\cdot|\rho|\rho\ t\cdots t \\
 \psi &::= \perp|\top|(\neg\psi)|(\psi \rightarrow \psi)|(\psi \wedge \psi)|(\psi \vee \psi)|(\psi \leftrightarrow \psi)|(\forall x.\psi)|(\exists x.\psi)|(\forall \rho.\psi)|(\exists \rho.\psi) \\
 &\quad | (t = t)|(\text{instance } t\ t)|(\text{subclass } t\ t)|(t \leq t)|(t < t)|(c\ s)
 \end{aligned}$$

The definition is mutually recursive since the term $\kappa x.\psi$ depends on the formula ψ . Of course, κ , $\forall$ and $\exists$ are binders. In practice, most occurrences of ρ are at the end of the spine. In some cases, however, extra arguments $t_1, \dots, t_n$ occur after the ρ . The idea is that ρ will be a list of arguments and $t_1, \dots, t_n$ will be appended to the end of that list. Note that at most one row variable can occur in a spine.

2.1 Implicit Type Guards

Properly parsing SUMO terms and formulas requires mechanisms for inferring implicit type guards for variables (interpreted conjunctively for κ and $\exists$ and via implication for $\forall$). Free variables in SUMO assertions are implicitly universally quantified and are restricted by inferred type guards, as described in [19]. In previous translations targeting first-order logic, relation and function variables are instantiated during the translation (treating the general statement quantifying over relations and functions as a macro to be expanded). Since the current translation will leave these as variables, we must also deal with type guards that are not known until the relation or function is instantiated.

2.2 Variable Arity Relations and Functions

Consider the SUMO relation `partition`, declared as follows:

```

(instance partition Predicate)
(instance partition VariableArityRelation)
(domain partition 1 Class)
(domain partition 2 Class)

```

The last three items indicate that `partition` has variable arity with at least 2 arguments, both of which are intended to be classes. If there are more than 2 arguments, the remaining arguments are also intended to be classes. In general, the extra optional arguments of a variable arity relation or function are intended to have the same domain as the last required argument. We will translate `partition` to a set that encodes not only when the relation should hold, but also its domain information, its minimum arity and whether or not it is variable arity.

Two other variable arity relations (with the same arity and type information as `partition`) are `exhaustiveDecomposition` and `disjointDecomposition`. The following is an example of a SUMO-K assertion relating these concepts:

$$\forall \rho. \text{partition } \rho \rightarrow \text{exhaustiveDecomposition } \rho \wedge \text{disjointDecomposition } \rho.$$

Previous translations to first-order logic expanded this assertion into several facts for different possible arities (using different predicates `partition3`, `partition4`, etc.), up to some limit. The following is an example of a partition occurring in `Merge.kif`⁵ with 6 arguments:

```
(partition Word Noun Verb Adjective Adverb ParticleWord)
```

From this one should be able to infer the following query:

Example 1 (wordex).

```
(query (exhaustiveDecomposition
        Word Noun Verb Adjective Adverb ParticleWord))
```

However, the corresponding first-order problem will not be provable unless the limit on the generated arity is at least 6. Our translation into set theory will free us from the need to know such limits in advance.

2.3 Quantification over Relations

`Merge.kif` includes assertions that quantify over relations. The following is an example of such an assertion:

```
(=>
  (and
    (subrelation ?REL1 ?REL2)
    (instance ?REL1 Predicate)
    (instance ?REL2 Predicate)
    (?REL1 @ROW))
  (?REL2 @ROW))
```

In previous first-order translations such assertions are instantiated with all R and R' where $(\text{subrelation } R \ R')$ is asserted. One of the 35 problems from [21] (TQG22) makes use of the SUMO assertion that `son` is a subrelation of `parent` and the macro expansion style of first-order translation is sufficient to handle this example. However, the macro expansion approach is insufficient to handle hypothetical subrelation assertions. The following is an example of a query creating a hypothetical subrelation assertion:

⁵ `Merge.kif` is the main SUMO ontology file. While `Merge.kif` evolves over time, we work with a fixed version of the file from January 2023. Latest versions of it and all the other files that make up SUMO are available at <https://github.com/ontologyportal/sumo>.

Example 2 (TQG22alt4).

```
(query (=> (exists (?X) (employs ?X ?X))
        (not (subrelation employs uses))))
```

During the process of answering this query we will assume `employs` is a subrelation of `uses` and then must instantiate the general assertion about subrelations with `employs` and `uses`. Our translation to set theory will permit this.

2.4 Kappa Binders

One of the 35 queries from [21] (TQG27) has the following local assumption making use of a κ -binder.

Example 3. The example TQG27 includes three assertions:

(A1) instance Planet Class, (A2) subclass Planet AstronomicalBody, and (the one with a κ -binder) (A3) instance o (κp .instance p Planet $\wedge$ attribute p Earthlike). Informally, one can read (A3) as $o \in \{p | p \text{ is an Earthlike planet}\}$. The query is (Q) instance o Planet.

The query should easily follow by eliminating the κ -abstraction. The first-order problem generated in [21] drops the assumption with the κ -abstraction (A3), making the problem unlikely to be provable (at least not for the intended reason). Our translation to set theory will handle κ -binders and the translation of this problem will be provable in the set theory.

2.5 Real Arithmetic

Six of the 35 examples from [21] involve some real arithmetic. Two simple example queries are the following:

Example 4 (TQG3).

```
(instance Number3-1 NonnegativeRealNumber)
(query (not (instance Number3-1 NegativeRealNumber)))
```

Example 5 (TQG11). (query (equal 12 (MultiplicationFn 3 4)))

For the sake of brevity we represent the first problem as having one local constant n , one local assumption instance n Nonneg and the query (conjecture) $\neg(\text{instance } n \text{ Neg})$. We will translate signed rationals with a finite decimal expansion to real numbers represented as sets.⁶ We will also translate `Real` to be equal to the set of reals $\mathbb{R}$. Furthermore we translate the operations $+$, $-$, $*$, $/$, $<$ and $\leq$ to have the appropriate meaning when applied to two reals.⁷ We then translate

⁶ We use a fixed construction of the reals, but the details of this are not relevant here.

⁷ To be more precise, we are using a specific set of reals constructed in the higher-order set theory, and operations (e.g., multiplication) are the expected set-theoretic operations on that set of reals. For simplicity, our set-theoretic division is a total function returning 0 when the denominator is 0.

Neg to $\{x \in \mathbb{R} \mid x < 0\}$ and **Nonneg** to $\{x \in \mathbb{R} \mid 0 \leq x\}$. Using the properties of the set-theoretic encoding, the translated queries above are set-theoretic theorems.

In addition to direct uses of arithmetic as in the examples above, arithmetic is also often used to check type guard information. This is due to the fact that a spine like $t_1 t_2 \rho$ will use subtraction to determine that under some constraints the i^{th} element of the corresponding list will be the $(i - 2)^{\text{nd}}$ element of the list interpreting ρ .

3 Translation of SUMO-K to Set Theory

3.1 High Level Overview: Sets, Terms, Spines and Formulas

Our translation maps terms t to sets. The particular set theory we use is *higher-order Tarski-Grothendieck* as described in [4].⁸ The details of this set theory are not important here. We only note that we have $\in, \subseteq$ (which will be used to interpret SUMO's **instance** and **subclass**) and that we have the ability to λ -abstract variables to form terms at higher types. The main types of interest are ι (the base type of sets), o (the type of propositions), $\iota \rightarrow \iota$ (the type of functions from sets to sets) and $\iota \rightarrow o$ (the type of predicates over sets).

Terms: When we say SUMO terms t are *translated to sets*, we mean they are translated to terms of type ι in the higher-order set theory.

Spines: Spines s are essentially lists of sets (of varying length). We translate them as functions that encode finite sequences. These functions are formally of the general type $\iota \rightarrow \iota$. However, we only use them when restricted to natural numbers, i.e., arguments $n \in \omega$ (where ω is the set of finite ordinals). We also maintain the invariant that the function returns the empty set on all but finitely many $n \in \omega$. An auxiliary function **listset** : $(\iota \rightarrow \iota) \rightarrow \iota$ gives a set-theoretic representation of the list by restricting its domain to ω .⁹

Tagging, Untagging, Length: To avoid confusion with the empty set being on a list, we tag elements of lists to ensure they are nonempty. Let $\mathsf{l} : \iota \rightarrow \iota$ be such a *tagging function* (injective on the universe of sets) and $\mathsf{U} : \iota \rightarrow \iota$ be an *untagging function*. We then define $\mathsf{nil} : \iota \rightarrow \iota$ to be constantly $\emptyset$ and $\mathsf{cons} : \iota \rightarrow (\iota \rightarrow \iota) \rightarrow \iota \rightarrow \iota$ to take a set x and a list l to the function mapping 0 to $\mathsf{l} x$ and $i + 1$ to $l i$ for $i \in \omega$. We also define a function $\mathsf{len} : (\iota \rightarrow \iota) \rightarrow \iota$ by $\lambda l. \{i \in \omega \mid l i \neq \emptyset\}$ ¹⁰ giving us the length of the list (assuming it is a list). Informally, a spine $t_0 \cdots t_{n-1}$ is thus a function taking i to $\mathsf{l}(t'_i)$ for each $i \in \{0, \dots, n-1\}$ where t'_i is the set-theoretic value of t_i and l the tagging function.

⁸ Tarski-Grothendieck is a set theory in which there are universes modeling ZFC set theory. These set-theoretic universes should not be confused with the universe of discourse **Univ1** introduced below.

⁹ We include **listset** since sometimes a list needs to be considered as a set.

¹⁰ Note that by design this set is the finite ordinal giving the length of the list.

Formulas: The translation of a SUMO formula ψ can be thought of either as a set (which should be one of the sets 0 or 1) or as a proposition. We also sometimes coerce between type ι and o by considering the sets 0 and 1 to be sets corresponding to false and true. Let $P : \iota \rightarrow o$ be $\lambda X. \emptyset \in X$ and let $B : o \rightarrow \iota$ be $\lambda p. \text{if } p \text{ then } 1 \text{ else } 0$. We use these functions as coercions between ι and o .

3.2 Motivating Examples

Before describing the translation in more detail, we give a few more simple examples to explain various aspects of the translation and motivate our choices.

Univ1 and Kappa: Let Univ1 be a set. This set is intended to be a *universe of discourse* in which most (but not all) targets of interpretation for t will live. Specifically, we will map the SUMO-type `Class` to the set $\wp \text{ Univ1}$ (the power set of the universe). We take all SUMO-types except the four special cases `Class`, `SetOrClass`, `Abstract` and `Entity` to be sets in $\wp \text{ Univ1}$. Consequently, if a SUMO object is an instance of some class other than `Class`, `SetOrClass`, `Abstract` and `Entity`, we will know that the object is a member of Univ1. Due to this we choose to translate κ -binders using simple separation bounded by Univ1. Reconsidering TQG27 discussed in Sect. 2.4 we translate instance o ($\kappa p. \text{instance } p \text{ Planet} \wedge \text{attribute } p \text{ Earthlike}$) to a set-theoretic proposition¹¹ of the form $o \in \{p \in \text{Univ1} \mid \dots p \in \text{PLANET} \wedge \dots\}$ (only partially specified at the moment).¹² From this set-theoretic proposition we can easily derive $o \in \text{PLANET}$ to solve the set-theoretic version of TQG27.

Variable Arity and Type Guards: As mentioned above, `partition` is a variable arity relation of at least arity 2 where every argument must be of SUMO-type `Class`. We will translate `partition` to a set PA containing multiple pieces of information. The behavior of PA as a relation is captured by the results one obtains by applying it to a set encoding a list of sets (via a set-theoretic operation $\text{ap} : \iota \rightarrow \iota \rightarrow \iota$). We can apply an abstract function $\text{arity} : \iota \rightarrow \iota$ to obtain the minimum arity of PA. We can apply an abstract predicate $\text{vararity} : \iota \rightarrow o$ to encode that PA has variable arity. Likewise we can apply an abstract $\text{domseq} : \iota \rightarrow \iota \rightarrow \iota$ to PA and an $i \in \omega$ to recover the intended domain of argument i of PA. These extra pieces of information are important to determine type guards in the presence of function and relation arguments.

In the specific case of `partition` the translation yields a set PA such that $\text{arity PA} = 2$, vararity PA is true and for $i \in \{0, 1, 2\}$, $\text{domseq PA } i = \wp \text{ Univ1}$. The value of $\text{domseq PA } 2$ determines the intended domain of all remaining (optional) arguments of the relation. (Note that SUMO indexes the first argument by 1 while in the set theory the first argument is indexed by 0.) The SUMO assertion

`(partition Word Noun Verb Adjective Adverb ParticleWord)`

¹¹ A set-theoretic proposition is a closed formula in the language of higher-order set theory [4].

¹² Note that the SUMO constant is `Planet` while its translated set-theoretic counterpart is `PLANET`.

translates to the set-theoretic statement¹³

P (ap PA (listset (cons Word (cons Noun (cons Verb (cons Adjective
(cons Adverb (cons ParticleWord nil)))))))).

Recall the SUMO-K assertion

$\forall \rho. \text{partition } \rho \rightarrow \text{exhaustiveDecomposition } \rho \wedge \text{disjointDecomposition } \rho.$

In this case the translation also generates type guards for the row variable ρ . Let PA, ED and DD be the sets corresponding to the SUMO constants `partition`, `exhaustiveDecomposition` and `disjointDecomposition`. Essentially, the assertion should only apply to ρ when ρ has at least length 2 and every entry is a (tagged) class. The translated set-theoretic statement (with type guards) is

$$\begin{aligned} \forall \rho : \iota \rightarrow \iota. \text{dom_of } (\text{vararity PA}) (\text{arity PA}) (\text{domseq PA}) \rho \\ \rightarrow \text{dom_of } (\text{vararity ED}) (\text{arity ED}) (\text{domseq ED}) \rho \\ \rightarrow \text{dom_of } (\text{vararity DD}) (\text{arity DD}) (\text{domseq DD}) \rho \\ \rightarrow P (\text{ap PA } \rho) \rightarrow P (\text{ap ED } \rho) \wedge P (\text{ap DD } \rho) \end{aligned}$$

The statement above makes use of a new definition: $\text{dom_of} : o \rightarrow \iota \rightarrow (\iota \rightarrow \iota) \rightarrow (\iota \rightarrow \iota) \rightarrow o$. The first argument of dom_of is a proposition encoding whether or not the function or relation is variable arity. In this case, all three of the propositions are variable arity (with the same typing information for all three). In the variable arity case $\text{dom_of } \top n D \rho$ is defined to be $\text{dom_of_varar } n D \rho$ where $\text{dom_of_varar} : \iota \rightarrow (\iota \rightarrow \iota) \rightarrow (\iota \rightarrow \iota) \rightarrow o$, n is the minimum arity, D is the list of domain information and ρ is the list we are requiring to satisfy the guard. $\text{dom_of_varar } n D \rho$ is defined to hold if the following three conditions hold:

1. $n \subseteq \text{len } \rho$ (ρ has at least length n)
2. $\forall i \in n, \cup (\rho i) \in D i$ and
3. $\forall i \in \text{len } \rho, n \subseteq i \rightarrow \cup (\rho i) \in D n$.

For fixed arity, dom_of is defined via a simpler dom_of_fixedar condition.

Another SUMO assertion about partitions is

(=>
(partition ?SUPER ?SUB1 ?SUB2)
(partition ?SUPER ?SUB2 ?SUB1))

In this case there are three standard (nonrow) variables needing type guards in the translation. Roughly speaking, `domseq PA` has the information we need, but in general we must modify it to be appropriate for variable arity relations. For this reason $\text{domseqm} : \iota \rightarrow \iota \rightarrow \iota$ is defined to be

$\lambda r i. \text{if vararity } r \text{ then domseq } r \text{ (if } i \in \text{arity } r \text{ then } i \text{ else arity } r) \text{ else domseq } r i.$

¹³ Note that we omit parentheses via the usual convention that implication is right associative, i.e., $\phi \rightarrow \psi \rightarrow \xi$ means $\phi \rightarrow (\psi \rightarrow \xi)$. Note also this is logically equivalent to $\phi \wedge \psi \rightarrow \xi$.

The translated statement is

$$\begin{aligned}
 \forall XYZ. X \in \text{domseqm PA } 0 \rightarrow Y \in \text{domseqm PA } 1 \rightarrow Z \in \text{domseqm PA } 2 \\
 \rightarrow Z \in \text{domseqm PA } 1 \rightarrow Y \in \text{domseqm PA } 2 \\
 \rightarrow \text{P}(\text{ap PA } (\text{cons } X (\text{cons } Y (\text{cons } Z \text{ nil})))) \\
 \rightarrow \text{P}(\text{ap PA } (\text{cons } X (\text{cons } Z (\text{cons } Y \text{ nil}))))).
 \end{aligned}$$

A simpler translation for handling type guards in this example could avoid the use of `dom_of` and `domseqm` and instead look up the arity and typing information for `partition`, etc. This translation would not work in general since SUMO assertions quantify over relations, in which case the particular type guards are not known until the relation variables are instantiated. Consider the SUMO-K formula

$$\forall R_1 R_2. \forall \rho. \text{subrelation } R_1 R_2 \wedge \text{instance } R_1 \text{ Predicate} \rightarrow \text{instance } R_2 \text{ Predicate} \\
 \rightarrow R_1 \rho \rightarrow R_2 \rho.$$

This translates to the set-theoretic proposition

$$\begin{aligned}
 \forall R_1 R_2 : \iota. \forall \rho : \iota \rightarrow \iota. R_1 \in \text{domseqm SR } 0 \rightarrow R_2 \in \text{domseqm SR } 1 \rightarrow \\
 R_1 \in E \rightarrow R_2 \in E \rightarrow \text{dom_of } (\text{vararity } R_1) (\text{arity } R_1) \rho \\
 \rightarrow \text{dom_of } (\text{vararity } R_2) (\text{arity } R_2) \rho \\
 \rightarrow \text{P } (\text{ap SR } (\text{cons } R_1 (\text{cons } R_2 \text{ nil}))) \wedge R_1 \in \text{PR} \rightarrow R_2 \in \text{PR} \rightarrow \text{P } (\text{ap } R_1 \rho) \\
 \rightarrow \text{P } (\text{ap } R_2 \rho)
 \end{aligned}$$

where `E`, `SR` and `PR` are the sets corresponding to the SUMO constants `Entity`, `subrelation` and `Predicate`. Here the type guards on ρ depend on R_1 and R_2 . Two special cases are the type guards $R_i \in E$ which are derived from the use of R_i as the first argument of `instance`.

3.3 The Translation

We now describe the translation itself. A first pass through the SUMO files given records the typing information from `domain`, `range`, `domainsubclass`, `rangesubclass` and `subrelation` assertions. A finite number of secondary passes determines which names will have variable arity (either due to a direct assertion or due to being inferred to be in a variable arity class).¹⁴

The final pass translates the assertions, and this is our focus here. Each SUMO-K assertion is a SUMO-K formula φ which may have free variables in it. Thus if we translate the SUMO-K formula φ into the set-theoretic proposition φ' , then the translated assertion will be

$$\forall x_1 \dots x_n. G_1 \rightarrow \dots G_m \rightarrow \varphi'$$

where $x_1, \dots, x_n$ are the free variables in φ and $G_1, \dots, G_m$ are the type guards for these free variables. Note that some of these free variables may be for spine

¹⁴ In practice with the current `Merge.kif` file, a single secondary pass suffices, but in general one might need an extra pass to climb the class hierarchy.

variables (i.e., row variables) and may have type $\iota \rightarrow \iota$. Such variables may also have type guards.

SUMO-K variables x translate to themselves where after translation x is a variable of type ι (ranging over sets). For SUMO-K constants c we choose a name c' and declare this as having type ι . Rational numbers q with a finite decimal expansion are translated to the set calculating the quotient of the base ten numerator divided by the appropriate power of 10. For example, 11.2 would be translated to the term $1 * 10^2 + 1 * 10 + 2$ divided by 10 (where 1, 2 and 10 are the usual finite ordinals and exponentiation by finite ordinals is defined by recursion). When a variable or constant is applied to a spine we translate the spine and use **ap**. As mentioned in Sect. 2.5 Real is translated to the set $\mathbb{R}$, Neg is translated to $\{x \in \mathbb{R} | x < 0\}$ and Nonneg is translated to $\{x \in \mathbb{R} | 0 \leq x\}$. The other arithmetical constructs are translated to sets, but we assume special properties such as

$$\forall xy \in \mathbb{R}. \mathbf{ap} \text{ ADD } (\mathbf{cons} \ x \ (\mathbf{cons} \ y \ \mathbf{nil})) = x + y,$$

$$\forall xy \in \mathbb{R}. \mathbf{ap} \text{ MULT } (\mathbf{cons} \ x \ (\mathbf{cons} \ y \ \mathbf{nil})) = x \cdot y$$

and

$$\forall xy \in \mathbb{R}. \mathbf{P} \ (\mathbf{ap} \ (\mathbf{LESSTHAN} \ (\mathbf{cons} \ x \ (\mathbf{cons} \ y \ \mathbf{nil})))) = (x < y).$$

- $(x \ s)$ translates to $(\mathbf{ap} \ x \ (\mathbf{listset} \ s'))$ where s' is the result of translating the SUMO-K spine s .
- $(c \ s)$ translates to $(\mathbf{ap} \ c' \ (\mathbf{listset} \ s'))$ where s' is the result of translating the SUMO-K spine s and c' is the chosen set as a counterpart to the SUMO-K constant c . Arithmetical operations are handled the same way.

The only remaining case for terms is κ binder terms.

- We translate $(\kappa x. \psi)$ to

$$\{x \in \mathbf{Univ1} \mid G_1 \wedge \dots \wedge G_m \wedge \psi'\}$$

where $G_1, \dots, G_m$ are generated type guards for x and ψ' is the result of translating the SUMO-K formula ψ to a set-theoretic proposition. Note that x ranges over $\mathbf{Univ1}$.

The translations of spines is relatively straightforward, but a few points are worth mentioning.

- The SUMO-K spine $(t \ s)$ is translated to the list one gets by applying $\mathbf{cons}$ to $I \ t'$ onto s' where t' is the translation of t and s' is the translation of s .
- A spine variable ρ is translated to itself (a variable of type $\iota \rightarrow \iota$).
- In the case $\rho \ t_1 \ \dots \ t_n$ we translate ρ to itself (a variable of type $\iota \rightarrow \iota$) and translate each t_i to a set t'_i and return the function that returns $\rho \ j$ given $j < \mathbf{len} \ \rho$ and returns t'_i given $\mathbf{len} \ \rho + i$ (appending the two lists).
- The empty spine is translated to $\mathbf{nil}$.

We consider each case of a SUMO-K formula. The usual logical operators are translated as the corresponding operators:

- $\perp$ and $\top$ translate simply to $\perp$ and $\top$.
- $(\neg \psi)$ translates to $\neg \psi'$ where ψ is a SUMO-K formula which translates to the set-theoretic proposition ψ' .
- $(\psi \rightarrow \xi)$ translates to $\psi' \rightarrow \xi'$ where ψ and ξ are SUMO-K formulas translate to the set-theoretic propositions ψ' and ξ' .
- $(\psi \leftrightarrow \xi)$ translates to $\psi' \leftrightarrow \xi'$ where ψ and ξ are SUMO-K formulas translate to the set-theoretic propositions ψ' and ξ' .
- Theoretically, $\psi \wedge \xi$ translates to $\psi' \wedge \xi'$. Practically speaking in SUMO-K conjunction is n -ary so it is more accurate to state that (**and** $\psi_1 \dots \psi_n$) translates to $\psi'_1 \wedge \dots \wedge \psi'_n$ where $\psi_1, \dots, \psi_n$ are SUMO-K formulas translate to the set-theoretic propositions $\psi'_1, \dots, \psi'_n$.
- Again, theoretically $\psi \vee \xi$ translates to $\psi' \vee \xi'$. Practically, (**or** $\psi_1 \dots \psi_n$) translates to $\psi'_1 \vee \dots \vee \psi'_n$ where $\psi_1, \dots, \psi_n$ are SUMO-K formulas translate to the set-theoretic propositions $\psi'_1, \dots, \psi'_n$.
- Theoretically, $\forall x. \psi$ translates to $\forall x. G_1 \rightarrow \dots \rightarrow G_m \rightarrow \psi'$ where ψ' is the result of translating ψ and $G_1, \dots, G_m$ are the generated type guards for x . Practically speaking, SUMO-K allows several variables to be universally quantified at once, so it is more accurate to say (**forall** $(x_1 \dots x_n) \psi$) translates to $\forall x_1 \dots x_n. G_1 \rightarrow \dots \rightarrow G_m \rightarrow \psi'$ where $x_1, \dots, x_n$ are variables, $G_1, \dots, G_m$ are the generated type guards for these variables and ψ' is the set-theoretic proposition obtained by translating ψ . That is, each G_i is a type guard induced by one of the variables x_j , with all the guards computed for the n variables simultaneously. While we could combine the guards into a single conjunction, we do not.
- $\forall \rho. \psi$ is translated similarly, but with type guards for the row variable ρ .
- Again, theoretically $\exists x. \psi$ translates to $\exists x. G_1 \wedge \dots \wedge G_m \wedge \psi'$, where ψ' is the set-theoretic proposition obtained by translating the SUMO-K formula ψ , but generalized to handle quantifying multiple variables.
- $\exists \rho. \psi$ is translated similarly, but with type guards for the row variable ρ .
- $(t_1 = t_2)$ translates to $t'_1 = t'_2$ where t_1 and t_2 are SUMO terms which translate to sets t'_1 and t'_2 .

We use set membership and inclusion to interpret **instance** and **subclass**.

- (**instance** $t_1 \ t_2$) translates to $t'_1 \in t'_2$ where t_1 and t_2 are SUMO terms which translate to sets t'_1 and t'_2 .
- (**subclass** $t_1 \ t_2$) translates to $t'_1 \subseteq t'_2$ where t_1 and t_2 are SUMO terms which translate to sets t'_1 and t'_2 .

4 Interactive Proofs of Translated SUMO Queries

The motivating set of examples were the 35 example queries from [21], now expanded¹⁵. Six of the original examples involve temporal reasoning. We omit

¹⁵ <https://github.com/ontologyportal/sumo/tree/master/tests>.

these for the moment, leaving a future translation to handle temporal and modal reasoning. 9 questions involve too many arguments for the existing first-order translation with macro expansion to work, but which are handled by our new translation. Among the remaining problems, 5 require some arithmetical reasoning, which use preexisting translations to standard first-order logic (FOF) and to an extension of first-order logic with arithmetic (TFF). For the remaining problems, the results of (at least) 5 were still not provable by the ATPs Vampire or E within a 600 s timeout.

We carefully looked at the set-theoretic translation of 13 of the problems that were too difficult for first-order provers (for any of the above reasons other than the use of temporal or modal reasoning). We either did an interactive proof or found slight modifications of the problem that could be interactively proven. The interactive proofs were done in Megalodon (the successor to the Egal system [4]). One advantage of having such a translation is the ability to attempt interactive proofs and recognize what may be missing from Merge.kif or the original query. We also did interactive proofs of 4 problems that the first-order provers could prove. We additionally included the 6 problems dealing with variable arity and row variables (e.g., Example 1). In total we have 23 SUMO-K queries translated to set-theoretic statements that have been interactively proven. We briefly describe some of the interactive proofs here.

An example with a particularly simple proof is TQG27 (Example 3), the example with a κ -binder. The assertion with the κ -binder translates to the set-theoretic proposition

$$o \in \{p \in \text{Univ1} \mid p \in E \wedge p \in \text{domseqm attribute } 0 \wedge p \in \text{Planet} \\ \wedge P(\text{ap attribute (listset (cons } p \text{ (cons Earthlike nil))}))\}.$$

The query translates simply to $o \in \text{Planet}$.

When interactively proving the translated query in Megalodon, we are free to use statements coming from three sources: set-theoretic propositions already previously proven in Megalodon (or are axioms of Tarski-Grothendieck), propositions resulting from the translation of formulas in Merge.kif, and propositions resulting from translating formulas local to the example. In this case we only need two propositions: the translated formula local to the example given above and one known set-theoretic proposition of the form:

$$\forall X : \iota. \forall P : \iota \rightarrow o. \forall x : \iota. x \in \{x \in X \mid P x\} \rightarrow x \in X \wedge P x.$$

From the two propositions we easily obtain the conjunction

$$o \in \text{Univ1} \wedge o \in E \wedge o \in \text{domseqm attribute } 0 \wedge o \in \text{Planet} \\ \wedge P(\text{ap attribute (listset (cons } o \text{ (cons Earthlike nil))})).$$

After this first step, a series of steps eliminate the conjunctions until we have the desired conjunct $o \in \text{Planet}$.

Another relatively simple example is TQG11 (Example 5) in which we must essentially prove 12 is $3 \cdot 4$. To be more precise we must prove

$$1 \cdot 10 + 2 = \text{ap MULT (listset (cons 3 (cons 4 nil)))}.$$

As mentioned in Sect. 3.3 the translation adds the proposition

$$\forall xy \in \mathbb{R}. \text{ap MULT (cons } x \text{ (cons } y \text{ nil))} = x \cdot y$$

which will be useful here. In the interactive proof, we first prove a claim that every natural number (finite ordinal) is a real number (i.e., $\omega \subseteq \mathbb{R}$, which is true for the representation of the reals being used). This claim is then used to prove $3 \in \mathbb{R}$ and $4 \in \mathbb{R}$. This allows us to reduce the main goal to proving $1 \cdot 10 + 2 = 3 \cdot 4$. This goal is then proven by an unsurprising sequence of rewrites using equations defining the behavior of $+$ and $\cdot$ on finite ordinals. (Many details are elided here, such as the fact that there are actually two different operations $+$, one on reals and one only on finite ordinals and that they provably agree on finite ordinals.)

We next consider the proof of the translation of Example 2. The set-theoretic proposition resulting from translating the query is

$$\begin{aligned} & (\exists x. x \in \text{domseqm employs } 0 \wedge x \in \text{domseqm employs } 1 \\ & \quad \wedge \text{P (ap employs (listset (cons } x \text{ (cons } x \text{ nil))))}) \\ & \rightarrow \neg \text{P (SR (listset (cons employs (cons uses nil))))}). \end{aligned}$$

We begin the interactive proof by proving the following sequence of claims:

1. $\text{len nil} = 0$.
2. $\forall X. \forall R : \iota \rightarrow \iota. \forall n. \text{nat_p } n \rightarrow \text{len } R = n \rightarrow \text{len (cons } X \text{ } R) = \text{ordsucc } n$.
3. $\forall y. \neg \text{vararity } y \rightarrow \forall i. \text{domseqm } y \text{ } i = \text{domseq } y \text{ } i$.
4. $\forall y. \neg \text{vararity } y \rightarrow \forall x i. x \in \text{domseq } y \text{ } i \rightarrow x \in \text{domseqm } y \text{ } i$.
5. $\forall X. \forall R : \iota \rightarrow \iota. \text{cons } X \text{ } R \text{ } 0 = \text{I } X$
6. $\forall n. \text{nat_p } n \rightarrow \forall X. \forall R : \iota \rightarrow \iota. \text{cons } X \text{ } R \text{ (ordsucc } n) = R \text{ } n$.

We can then rewrite `domseqm employs` into `domseq employs`. Starting the main body of the proof, we assume we have an x such that $x \in \text{domseq employs } 0$, $x \in \text{domseq employs } 1$ and $\text{P (ap employs (listset (cons } x \text{ (cons } x \text{ nil))))}$. We further assume $\text{P (SR (listset (cons employs (cons uses nil))))}$ and prove a contradiction. Using the translated Merge.kif type information from `employs` we can infer x is an autonomous agent and an object. Likewise we can infer `employs` is a predicate and a relation, and the same for `uses`. The contradiction follows from two claims: $\text{P (ap uses (cons } x \text{ (cons } x \text{ nil)))}$ and $\neg \text{P (ap uses (cons } x \text{ (cons } x \text{ nil)))}$.

We first prove $\text{P (ap uses (cons } x \text{ (cons } x \text{ nil)))}$. We locally let `ROW` be `cons } x \text{ (cons } x \text{ nil)}` and use the claims above prove from `ROW 0 = I } x`, `ROW 1 = I } x`, `U (ROW 0) = } x`, `U (ROW 1) = } x` and `len ROW = 2`. We can then essentially complete the subproof using the local assumptions

$$\text{P (ap employs (listset (cons } x \text{ (cons } x \text{ nil))))}$$

and

$$\text{P (SR (listset (cons employs (cons uses nil))))}$$

along with the translation of the following Merge.kif formula:

```
(=>
  (and
    (subrelation ?REL1 ?REL2)
    (instance ?REL1 Predicate)
    (instance ?REL2 Predicate)
    (?REL1 @ROW))
  (?REL2 @ROW))
```

To complete the contradiction we prove $\neg P$ (`ap uses (cons x (cons x nil))`). The three most significant Merge.kif formulas whose translated propositions are used in the subproof are:

```
(instance uses AsymmetricRelation)
(subclass AsymmetricRelation IrreflexiveRelation)
(=>
  (instance ?REL IrreflexiveRelation)
  (forall (?INST)
    (not
      (?REL ?INST ?INST))))
```

That is, Merge.kif declares that `uses` is an asymmetric relation, every asymmetric relation is an irreflexive relation, and that irreflexive relations have the expected property of irreflexivity.

5 ATP Problem Set

After interactively proving the 23 problems, we created TH0¹⁶ problems restricted to the axioms used in the proof. This removes the need for the higher-order ATP to do premise selection. Additionally we used Megalodon to analyze the interactive proof to create a number of subgoal problems for ATPs – ranging from the full problem (the initial goal to be proven) to the smallest subgoals (completed by a single tactic). For example, the interactive proofs of Examples 1, 2 and 5 generate 415, 322 and 100 TH0 problems, respectively. In total analysis of the interactive proofs yields 4880 (premise-minimized) TH0 problems for ATPs. In Table 1 we give the results for several higher-order automated theorem provers (Leo-III [24], Vampire [13], Lash [3], Zipperposition [27], E [22]), given a 60s timeout.

6 Future Work

The primary plan to extend the translation is to include temporal and modal operators. SUMO includes many modal operators including necessity, possibility,

¹⁶ TH0 was introduced as THF0 in [2] as a core language for representing typed higher-order formulas (in the sense of Church’s simple type theory) for automated theorem provers.

Table 1. Number of Subgoals Proven Automatically in 60 s

Problem	Subgoals	Zipperposition	Vampire	E	Lash	Leo-III
TQG1	50	50 (100%)	50 (100%)	50 (100%)	50 (100%)	50 (100%)
TQG3	20	20 (100%)	20 (100%)	14 (70%)	20 (100%)	8 (40%)
TQG7	195	188 (96%)	185 (95%)	180 (92%)	160 (82%)	158 (81%)
TQG9	19	19 (100%)	19 (100%)	19 (100%)	19 (100%)	19 (100%)
TQG10	112	112 (100%)	112 (100%)	100 (89%)	58 (52%)	96 (86%)
TQG11	100	76 (76%)	39 (39%)	67 (67%)	45 (45%)	13 (13%)
TQG19	37	34 (92%)	22 (59%)	20 (54%)	37 (100%)	11 (30%)
TQG20	41	34 (83%)	22 (54%)	20 (49%)	41 (100%)	13 (32%)
TQG21	207	154 (74%)	150 (72%)	143 (69%)	101 (49%)	56 (27%)
TQG22alt3	319	246 (77%)	214 (67%)	193 (61%)	197 (62%)	136 (43%)
TQG22alt4	322	251 (78%)	218 (68%)	197 (61%)	201 (62%)	142 (44%)
TQG22	315	271 (86%)	224 (71%)	212 (67%)	201 (64%)	142 (45%)
TQG23	67	61 (91%)	67 (100%)	42 (63%)	51 (76%)	38 (57%)
TQG25alt1	910	652 (72%)	526 (58%)	580 (64%)	529 (58%)	246 (27%)
TQG27	7	7 (100%)	7 (100%)	7 (100%)	7 (100%)	7 (100%)
TQG28alt1	600	428 (71%)	386 (64%)	349 (58%)	261 (44%)	213 (36%)
TQG30	4	4 (100%)	4 (100%)	3 (75%)	4 (100%)	4 (100%)
TQG33	112	82 (73%)	83 (74%)	79 (71%)	85 (76%)	36 (32%)
TQG45	162	136 (84%)	131 (81%)	128 (79%)	106 (65%)	36 (22%)
TQG46	344	258 (75%)	215 (62%)	225 (65%)	163 (47%)	144 (42%)
TQG47	186	141 (76%)	113 (61%)	109 (59%)	93 (50%)	79 (42%)
TQG48	336	249 (74%)	234 (70%)	219 (65%)	184 (55%)	146 (43%)
wordex	415	315 (76%)	255 (61%)	236 (57%)	284 (68%)	143 (34%)
Total	4880	3788 (78%)	3296 (68%)	3192 (65%)	2897 (59%)	1936 (40%)

deontological operators (obligation and permission) and modalities for knowledge, beliefs and desires. Each modality can be modelled using Kripke style semantics [14] (possible worlds with an accessibility relation).

The following is an example of a SUMO formula in Merge.kif using modalities¹⁷ :

```
(=>
  (modalAttribute ?FORMULA Necessity)
  (modalAttribute ?FORMULA Possibility))
```

¹⁷ Note that SUMO embeds several different modalities that have different axiomatizations. Rather than assuming one particular modal logic axiomatization (S4, S5 etc.) by embedding different modal logics in higher-order logic we hope to determine if we can create a coherent system of axiomatizations while avoiding known paradoxes like the gentle murderer paradox.

The current translation simply skips these formulas as they are not in the SUMO-K fragment. If we only wanted to extend the translation to include necessity and possibility, we could change the translation to make the dependence on worlds explicit. The SUMO formula above could translate to the proposition

$$\forall w \in W. \forall \varphi : \iota \rightarrow \iota. (\forall v \in W. R \ w \ v \rightarrow P(\varphi \ v)) \rightarrow (\exists v \in W. R \ w \ v \wedge P(\varphi \ v)).$$

Here W is a set of worlds and R is an accessibility relation on W . Note that the translated formula variable has type $\iota \rightarrow \iota$ instead of type ι to make the dependence of the formula on the world explicit. In general, terms, spines and formulas would depend on a world w and in an asserted formula the world w would be universally quantified (ranging over W) as above.

If we took the approach above to model necessity and possibility, then to add deontic modalities later we would need a second set of worlds and accessibility relation. The translation of terms would then have type $\iota \rightarrow \iota \rightarrow \iota$ to account for the dependence on both kinds of worlds. In order to prevent needing to keep adding new dependencies for every modalities, our plan is to combine the sets of worlds and accessibility relations in an extensible way. Thus terms will translate to have type $\iota \rightarrow \iota$ essentially giving dependence on a single set encoding a sequence of worlds (where we are open ended about the length of the sequence). Using this idea, the SUMO formula above would translate to something like

$$\begin{aligned} \forall w \in (\Pi x \in X. W \ x). \forall \varphi : \iota \rightarrow \iota. (\forall v \in (\Pi x \in X. W \ x). R \ m \ w \ v \rightarrow P(\varphi \ v)) \\ \rightarrow (\exists v \in (\Pi x \in X. W \ x). R \ m \ w \ v \wedge P(\varphi \ v)) \end{aligned}$$

where X is an index set (where each $x \in X$ corresponds to a modality being interpreted), $m \in X$ is the specific index for necessity and possibility, $W \ x$ is the set of worlds for x , and $R \ x$ is a relation between $w, v \in \Pi x \in X. W \ x$ that holds if the x components satisfy the accessibility relation over $W \ x$ and the other components of w and v do not change. This allows us to model an arbitrary number of modalities using Kripke semantics while only carrying one world argument. Another advantage is that it minimizes the change to the translation of formulas in the SUMO-K fragment (without modalities). The only required change is to add a single dependence on w via a new argument and universally quantify over w if the formula is asserted.

We have already done some experiments with this approach and it shows promise. The previous experiments need to be extended to include changes that have occurred to obtain the SUMO-K translation described in the present paper. Once this is done, we must ensure that translated examples both with modalities and the examples in this paper without modalities are provable interactively. We plan to also test automated theorem provers on the subgoals obtained from the interactive proofs. Doing so with the 23 examples in this paper will give an indication how much more difficult the translated problems become if the Kripke infrastructure to handle modalities is included.

Another aspect of SUMO are modalities involving likelihood and probability. These cannot be modelled by Kripke semantics (as the modalities are not normal). We are experimenting with using neighborhood semantics to include these modalities.

7 Conclusion

We have described a translation from the SUMO-K fragment of SUMO into higher-order set theory. We have considered a number of examples that use aspects of SUMO-K that go beyond traditional first-order logic, namely variable arity functions and relations, row variables, term-level κ -binders and arithmetic. We have described a number of interactive proofs of translated queries and tested higher-order automated theorem provers on problems obtained by doing premise selection using the corresponding interactive proofs. This gives a set of problems for automated theorem provers that come from the area of “common sense reasoning,” an area quite different from the more common sources of formalized mathematics and program verification. On most of the examples, higher-order automated theorem provers cannot fully automatically prove the query, but they perform reasonably well on subgoal problems extracted from the interactive proofs. This gives an indication that the full problems (assuming premise selection) are not too far out of reach for current state of the art higher-order automated theorem provers.

Acknowledgments. This work was partially supported by the ERC-CZ project POSTMAN no. LL1902, Amazon Research Awards, EU ICT-48 2020 project TAILOR no. 952215 and the European Regional Development Fund under the Czech project AI&Reasoning with identifier CZ.02.1.01/0.0/0.0/15_003/0000466.

References

1. Benz Müller, C., Pease, A.: Higher-Order Aspects and Context in SUMO. In: Jos Lehmann, I.J.V., Bundy, A. (eds.) Special issue on Reasoning with context in the Semantic Web, vol. 12–13. Science, Services and Agents on the World Wide Web (2012)
2. Benz Müller, C., Rabe, F., Sutcliffe, G.: Thf0 - the core of the tptp language for higher-order logic. In: Armando, A., Baumgartner, P., Dowek, G. (eds.) Automated Reasoning, pp. 491–506. Springer, Berlin Heidelberg, Berlin, Heidelberg (2008)
3. Brown, C.E., Kaliszyk, C.: Lash 1.0 (system description). In: Blanchette, J., Kovács, L., Pattinson, D. (eds.) Automated Reasoning - 11th International Joint Conference, IJCAR 2022, Haifa, Israel, August 8–10, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13385, pp. 350–358. Springer (2022)
4. Brown, C.E., Pał, K.: A Tale of Two Set Theories. In: Kaliszyk, C., Brady, E., Kohlhas, A., Sacerdoti Coen, C. (eds.) CICM 2019. LNCS (LNAI), vol. 11617, pp. 44–60. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-23250-4_4
5. Chvalovský, K., Jakubuv, J., Suda, M., Urban, J.: ENIGMA-NG: efficient neural and gradient-boosted inference guidance for E. In: Fontaine, P. (ed.) Automated Deduction - CADE 27–27th International Conference on Automated Deduction, Natal, Brazil, August 27–30, 2019, Proceedings. Lecture Notes in Computer Science, vol. 11716, pp. 197–215. Springer (2019). https://doi.org/10.1007/978-3-030-29436-6_12

6. Jakubuv, J., Chvalovský, K., Olsák, M., Piotrowski, B., Suda, M., Urban, J.: ENIGMA anonymous: Symbol-independent inference guiding machine (system description). In: Peltier, N., Sofronie-Stokkermans, V. (eds.) Automated Reasoning - 10th International Joint Conference, IJCAR 2020, Paris, France, July 1–4, 2020, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12167, pp. 448–463. Springer (2020). https://doi.org/10.1007/978-3-030-51054-1_29
7. Jakubuv, J., Urban, J.: ENIGMA: efficient learning-based inference guiding machine. In: Geuvers, H., England, M., Hasan, O., Rabe, F., Teschke, O. (eds.) Intelligent Computer Mathematics - 10th International Conference, CICM 2017, Edinburgh, UK, July 17–21, 2017, Proceedings. Lecture Notes in Computer Science, vol. 10383, pp. 292–302. Springer (2017). <https://doi.org/10.1007/978-3-319-62075-6>
8. Jakubuv, J., Urban, J.: Hammering Mizar by learning clause guidance. In: Harrison, J., O’Leary, J., Tolmach, A. (eds.) 10th International Conference on Interactive Theorem Proving, ITP 2019(September), pp. 9–12, 2019. Portland, OR, USA. LIPIcs, vol. 141, pp. 34:1–34:8. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2019). <https://doi.org/10.4230/LIPIcs.ITP.2019.34>
9. Kaliszyk, C., Urban, J., Vyskočil, J.: Automating Formalization by Statistical and Semantic Parsing of Mathematics. In: Ayala-Rincón, M., Muñoz, C.A. (eds.) ITP 2017. LNCS, vol. 10499, pp. 12–27. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-66107-0_2
10. Kaliszyk, C., Urban, J., Vyskočil, J.: Learning to parse on aligned corpora (rough diamond). In: Urban, C., Zhang, X. (eds.) Interactive Theorem Proving - 6th International Conference, ITP 2015, Nanjing, China, August 24–27, 2015, Proceedings. Lecture Notes in Computer Science, vol. 9236, pp. 227–233. Springer (2015). <https://doi.org/10.1007/978-3-319-22102-1>
11. Kaliszyk, C., Urban, J., Vyskočil, J., Geuvers, H.: Developing corpus-based translation methods between informal and formal mathematics: Project description. In: Watt, S.M., Davenport, J.H., Sexton, A.P., Sojka, P., Urban, J. (eds.) Intelligent Computer Mathematics - International Conference, CICM 2014, Coimbra, Portugal, July 7–11, 2014. Proceedings. LNCS, vol. 8543, pp. 435–439. Springer (2014). https://doi.org/10.1007/978-3-319-08434-3_34
12. Kovács, L., Voronkov, A.: First-Order Theorem Proving and Vampire. In: Sharygina, N., Veith, H. (eds.) CAV 2013. LNCS, vol. 8044, pp. 1–35. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39799-8_1
13. Kovács, L., Voronkov, A.: First-Order Theorem Proving and Vampire. In: Sharygina, N., Veith, H. (eds.) CAV 2013. LNCS, vol. 8044, pp. 1–35. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39799-8_1
14. Kripke, S.A.: Semantical analysis of modal logic i normal modal propositional calculi. *Mathematical Logic Quarterly* **9**, 67–96 (1963). <https://doi.org/10.1002/malq.19630090502>

15. Niles, I., Pease, A.: Toward a Standard Upper Ontology. In: Welty, C., Smith, B. (eds.) *Proceedings of the 2nd International Conference on Formal Ontology in Information Systems (FOIS-2001)*. pp. 2–9 (2001)
16. Pease, A.: *Ontology: A Practical Guide*. Articulate Software Press, Angwin, CA (2011)
17. Pease, A.: Arithmetic and inference in a large theory. In: *AI in Theorem Proving* (2019)
18. Pease, A.: Converting the Suggested Upper Merged Ontology to Typed First-order Form [arXiv:2303.04148](https://arxiv.org/abs/2303.04148) [cs.AI] (2023)
19. Pease, A., Schulz, S.: Knowledge Engineering for Large Ontologies with Sigma KEE 3.0. In: Demri, S., Kapur, D., Weidenbach, C. (eds.) *IJCAR 2014*. LNCS (LNAI), vol. 8562, pp. 519–525. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-08587-6_40
20. Pease, A., Sutcliffe, G., Siegel, N., Trac, S.: Large Theory Reasoning with SUMO at CASC. *AI Communications, Special issue on Practical Aspects of Automated Reasoning* **23**(2–3), 137–144 (2010)
21. Pease, A., Sutcliffe, G., Siegel, N., Trac, S.: Large theory reasoning with sumo at casc. *AI Commun.* **23**(2–3), 137–144 (2010). <https://doi.org/10.3233/AIC-2010-0466>
22. Schulz, S.: E - A Brainiac Theorem Prover. *AI Commun.* **15**(2–3), 111–126 (2002)
23. Schulz, S., Sutcliffe, G., Urban, J., Pease, A.: Detecting inconsistencies in large first-order knowledge bases. In: *Proceedings of CADE 26*. pp. 310–325. Springer (2017)
24. Steen, A., Benz Müller, C.: The higher-order prover leo-iii. CoRR abs/1802.02732 (2018), <https://arxiv.org/abs/1802.02732>
25. Sutcliffe, G.: TPTP, TSTP, CASC, etc. In: *Proceedings of the Second International Conference on Computer Science: Theory and Applications*. pp. 6–22. CSR'07, Springer-Verlag, Berlin, Heidelberg (2007), <https://dl.acm.org/citation.cfm?id=2391910.2391914>
26. Sutcliffe, G., Schulz, S., Claessen, K., Baumgartner, P.: The TPTP Typed First-Order Form with Arithmetic. In: Bjørner, N., Voronkov, A. (eds.) *LPAR 2012*. LNCS, vol. 7180, pp. 406–419. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-28717-6_32
27. Vukmirović, P., Bentkamp, A., Blanchette, J., Cruanes, S., Nummelin, V., Tournet, S.: Making higher-order superposition work. In: Platzner, A., Sutcliffe, G. (eds.) *Automated Deduction - CADE 28*, pp. 415–432. Springer International Publishing, Cham (2021)
28. Wang, Q., Kaliszyk, C., Urban, J.: First Experiments with Neural Translation of Informal to Formal Mathematics. In: Rabe, F., Farmer, W.M., Passmore, G.O., Youssef, A. (eds.) *CICM 2018*. LNCS (LNAI), vol. 11006, pp. 255–270. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-96812-4_22

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

